TRABAJO PRÁCTICO: MACHINE LEARNING & IA 🛡️💻

Implementación y Análisis de Clasificadores Bayesianos (Naive Bayes)

Ejercicio 6b: Detección Automática de Patrones de Grooming

📂 Información Académica

- 🏫 **Institución:** Instituto Tecnológico Beltrán (ISFT 197)
- 📊 **Carrera:** Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial
- ⚙️ **Cátedra:** Procesamiento de Aprendizaje Automático
- 👩‍🏫 **Docente:** Yanina Escudero
- 🗓️ **Fecha:** Octubre de 2026
- 👥 **Integrantes del Equipo (Grupo 4):**
  - Silvana Alejandra Gerez
  - Danilo Figueroa
  - Alejandra Bernasconi Lan

📌 Consigna del Ejercicio 6b

Contexto (En Equipo):

Una organización que gestiona una plataforma de mensajería quiere implementar un sistema automático para detectar posibles casos de *grooming* (acoso a menores en línea) a partir de mensajes de texto enviados por los usuarios.

Objetivo:

Entrenar un modelo de clasificación Naive Bayes con un conjunto de mensajes etiquetados como *grooming* o no *grooming*, evaluar su rendimiento y usarlo para identificar mensajes sospechosos en tiempo real.

In [ ]:
```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
import pandas as pd

############################
# Enunciado del Ejercicio  #
############################
# Contexto:
# Una organización que gestiona una plataforma de mensajería quiere implementar un sistema
# automático para detectar posibles casos de grooming (acoso a menores en línea) a partir de mens
# texto enviados por los usuarios.

# Objetivo:
# Entrenar un modelo de clasificación Naive Bayes con un conjunto de mensajes etiquetados
# como grooming o no grooming, evaluar su rendimiento y usarlo para identificar mensajes sospecho
# en tiempo real.
```

```
#########################
# Datos de entrenamiento#
#########################
data = {
    'text': [
        "¿Cuántos años tenés? No le digas a nadie que hablamos.",
        "Hola, ¿cómo estás? ¿Querés jugar algo online?",
        "Sos muy linda, no le muestres esto a tus papás.",
        "¿Terminaste la tarea de matemáticas?",
        "Podés mandarme una foto tuya, pero que sea solo para mí.",
        "¿Querés que te ayude con el juego que no podías pasar?",
        "No le contés a nadie que hablamos por acá, ¿sí?",
        "¿Te gustó la película de ayer?",
        "¿Estás solo en casa ahora?",
        "¿Querés hacer una videollamada para estudiar juntos?",
        "No le digas a tus padres que te escribí, es nuestro secreto.",
        "¿Cómo te fue en el examen de historia?",
        "Mandame una foto tuya, pero que nadie más la vea.",
        "¿Jugamos Minecraft esta tarde?",
        "Sos muy especial para mí, no le cuentes a nadie lo que hablamos.",
        "¿Querés que te pase los apuntes de biología?",
        "¿Podés mostrarme cómo estás vestida ahora?",
        "¿Tenés ganas de salir a andar en bici mañana?",
        "No hace falta que le digas a nadie que hablamos tanto.",
        "¿Querés que estudiemos juntos para el parcial?"
    ],
    'label': [
        'grooming', 'no grooming',
        'grooming', 'no grooming',
        'grooming', 'no grooming',
        'grooming', 'no grooming',
        'grooming', 'no grooming',
        'grooming', 'no grooming',
        'grooming', 'no grooming',
        'grooming', 'no grooming',
        'grooming', 'no grooming',
        'grooming', 'no grooming'
    ]
}

df = pd.DataFrame(data)

########################################
#     Preprocesamiento y modelo        #
# 1-Separar características y etiquetas#
########################################
X = df['text']
y = df['label']

# Convertir texto en vectores de frecuencia
vectorizer = CountVectorizer()
X_vectorized = vectorizer.fit_transform(X)

##########################################
# 2-Dividir en entrenamiento y prueba    #
##########################################
X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y, test_size=0.3, random_state=

########################################
# 3-Entrenar el clasificador Naive Bayes#
########################################
model = MultinomialNB()
model.fit(X_train, y_train)

################################
# 4-Evaluación del modelo
################################
y_pred = model.predict(X_test)

print("Reporte de clasificación:")
print(classification_report(y_test, y_pred))
```

```python
print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred))

# ###################################
# 5-Clasificación de nuevas noticias #
# ###################################

nuevas_noticias = [
    "¿Podés mandarme una foto tuya? No se la muestres a nadie.",
    "¿Jugamos Minecraft esta tarde?",
    "No le digas a tus papás que hablamos, ¿sí?",
    "¿Terminaste el trabajo práctico de historia?"
]

nuevas_vectorizadas = vectorizer.transform(nuevas_noticias)
predicciones = model.predict(nuevas_vectorizadas)

for noticia, prediccion in zip(nuevas_noticias, predicciones):
    print(f'Mensaje: "{noticia}" → Clasificación: {prediccion}')
```

```
Reporte de clasificación:
              precision    recall  f1-score   support

    grooming       1.00      1.00      1.00         2
 no grooming       1.00      1.00      1.00         4

    accuracy                           1.00         6
   macro avg       1.00      1.00      1.00         6
weighted avg       1.00      1.00      1.00         6

Matriz de confusión:
[[2 0]
 [0 4]]
Mensaje: "¿Podés mandarme una foto tuya? No se la muestres a nadie." → Clasificación: grooming
Mensaje: "¿Jugamos Minecraft esta tarde?" → Clasificación: no grooming
Mensaje: "No le digas a tus papás que hablamos, ¿sí?" → Clasificación: grooming
Mensaje: "¿Terminaste el trabajo práctico de historia?" → Clasificación: no grooming
```

📊 Informe e Interpretación de Resultados - Ejercicio 6b

- **1. Rendimiento General y Vectorización:** El modelo Naive Bayes Multinomial (`MultinomialNB`) procesó exitosamente la vectorización del corpus oficial de 20 mensajes mediante `CountVectorizer`, logrando aislar con absoluta precisión los patrones léxicos formales e informales de las interacciones cotidianas y académicas (`no grooming`) frente a los indicadores de riesgo, asedio y solicitud de ocultamiento (`grooming`).

- **2. Evaluación con Datos de Prueba (`train_test_split`):** La partición del conjunto de datos permitió evaluar el comportamiento predictivo del clasificador sobre muestras no presenciadas durante el entrenamiento. Los resultados reflejan un rendimiento óptimo:
  - **Exactitud (*Accuracy*):** `1.00` sobre el conjunto de prueba ($n = 6$).
  - **Matriz de Confusión (`[[2, 0], [0, 4]]`):** El modelo clasificó correctamente la totalidad de los casos, sin registrar falsos positivos ni falsos negativos, validando la solidez matemática de las probabilidades condicionales del teorema de Bayes para este dominio sensible.

- **3. Análisis de Nuevos Mensajes en Tiempo Real:**

- 📱 *"¿Podés mandarme una foto tuya? No se la muestres a nadie."*
  ➔ Resultado: 🚨 **GROOMING**
  (Nota técnica: El clasificador identifica con precisión el vector de riesgo compuesto por la exigencia de material íntimo)

- 🎮 *"¿Jugamos Minecraft esta tarde?"*
  ➔ Resultado: 🟢 **NO GROOMING**
  (Identificado correctamente dentro de los patrones léxicos habituales de recreación y ocio entre pares).

- ⚠️ *"No le digas a tus papás que hablamos, ¿sí?"*
  ➔ Resultado: 🚨 **GROOMING**
  (Nota técnica: El modelo detecta exitosamente la directiva de ocultamiento familiar, uno de los indicadores conductuales clave definidos en el corpus de entrenamiento).

- 📚 *"¿Terminaste el trabajo práctico de historia?"*
  ➔ Resultado: 🟢 **NO GROOMING**
  (Clasificado correctamente dentro de los parámetros habituales de colaboración académica y consulta escolar).